

ANOTACIONES JPA – REFERENCIA COMPLETA

B2

Anotación	Obligatoria	Descripción
@Entity	✓ Sí	Marca la clase como entidad mapeada a tabla
@Id	✓ Sí	Define el atributo como clave primaria
@GeneratedValue	Recomendada	Indica que el ID se genera automáticamente
@Table(name="x")	Opcional	Nombre personalizado de la tabla
@Column(name="x")	Opcional	Nombre personalizado de columna
@Column(nullable=false)	Opcional	Equivale a NOT NULL en la tabla
@Column(unique=true)	Opcional	Equivale a UNIQUE en la tabla
@Transient	Opcional	El atributo no se persiste en BD

ESTRATEGIAS DE GENERATEDVALUE

Estrategia	Descripción
IDENTITY	La BD genera el ID (AUTO_INCREMENT). Más usada con MySQL.
SEQUENCE	Usa una secuencia de BD (Oracle, PostgreSQL).
AUTO	El proveedor elige la estrategia según la BD.
TABLE	Tabla auxiliar para gestionar IDs. Poco usada.

PERSISTENCE.XML – PROPIEDADES CLAVE

Propiedad	Qué hace
jdbc.url	URL de conexión a la BD
jdbc.user	Usuario de la BD
jdbc.password	Contraseña de la BD
jdbc.driver	Claase del driver JDBC
hibernate.dialect	Adapta Hibernate al SGBD concreto
hbm2ddl.auto=update	Sincroniza tablas automáticamente
hbm2ddl.auto=create	Recrea tablas en cada inicio (borra datos)
hbm2ddl.auto=validate	Valida esquema sin modificar
show_sql=true	Muestra el SQL generado por Hibernate

X ERRORES COMUNES

- B2

No añadir constructor vacío `public Usuario() {}` → Hibernate no puede instanciar la entidad
- B2

Olvidar `getTransaction().begin()` antes de `persist()` → `TransactionRequiredException`
- B2

No hacer `commit()` → los cambios no se guardan aunque no haya error
- B2

Usar `hbm2ddl.auto=create` en producción → borra todos los datos en cada reinicio

CRUD COMPLETO CON JPA

B2

```
import jakarta.persistence.*;

public class Main {
    public static void main(String[] args) {

        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("miUnidad");
        EntityManager em = emf.createEntityManager();

        // CREATE – persist
        em.getTransaction().begin();
        Usuario u = new Usuario("Ana", "ana@mail.com");
        em.persist(u);
        em.getTransaction().commit();
        // → Hibernate genera INSERT

        // READ – find
        Usuario encontrado = em.find(Usuario.class, u.getId());
        System.out.println(encontrado.getNombre());
        // → Hibernate genera SELECT WHERE id=?

        // UPDATE – sin método update; JPA detecta cambios
        em.getTransaction().begin();
        encontrado.setNombre("Ana Actualizada");
        em.getTransaction().commit();
        // → Hibernate genera UPDATE automáticamente

        // DELETE – remove
        em.getTransaction().begin();
        em.remove(encontrado);
        em.getTransaction().commit();
        // → Hibernate genera DELETE

        em.close(); emf.close();
    }
}
```

DEPENDENCIAS MAVEN

B2

```
<dependency>
<groupId>jakarta.persistence</groupId>
<artifactId>jakarta.persistence-api</artifactId>
<version>3.1.0</version>
</dependency>
<dependency>
<groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-core</artifactId>
<version>6.4.0.Final</version>
</dependency>
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>8.3.0</version>
</dependency>
```

Ruta del archivo: `src/main/resources/META-INF/persistence.xml`

DAM · PROGRAMACIÓN · ORM / JPA

ORM y JPA con Hibernate

BLOQUES 1 Y 2 · CHULETARIO · TRÍPTICO A4

EQUIVALENCIA JAVA ↔ BASE DE DATOS



ARQUITECTURA JPA + HIBERNATE

```
// Tu código
em.persist(usuario)
↓
JPA API      + defines las reglas
↓
Hibernate    + ejecuta las operaciones
↓
INSERT INTO usuarios ...
↓
Base de datos MySQL
```

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq. — B1

ORM vs JDBC, problemas del mapeo manual, ventajas
- Interior cent. — B2

Entidad, anotaciones, EntityManager, transacciones
- Interior der. — B2

persistence.xml, ciclo de uso, operaciones internas
- Solapa —

CRUD completo real, dependencias Maven
- Contraportada —

Todas las anotaciones, estrategias de ID, errores comunes

JDBC VS ORM/JPA

JDBC manual SQL escrito a mano · ResultSet → objeto manual · Código repetitivo · Fuerte acoplamiento con BD	JPA/Hibernate SQL generado automáticamente · Objeto ↔ tabla automático · Código más limpio · Menos repetición
---	---

? EL PROBLEMA: DOS MUNDOS DISTINTOS B1

Java trabaja con **objetos**. Las bases de datos trabajan con **tablas**. Son mundos incompatibles: Java no puede guardar objetos directamente en BD, y la BD no entiende objetos.

MAPEO MANUAL CON JDBC – EL PROBLEMA

```
// Para leer un objeto hay que hacer todo esto:
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM usuarios WHERE id = ?");
stmt.setInt(1, 1);
ResultSet rs = stmt.executeQuery();
Usuario u = null;
if (rs.next()) {
    u = new Usuario();
    u.setId(rs.getInt("id"));
    u.setNombre(rs.getString("nombre"));
    u.setEmail(rs.getString("email"));
}
// -> Esto hay que repetirlo para CADA entidad
```

PROBLEMAS DEL ENFOQUE MANUAL

- **Código repetitivo** – SQL + conexión + mapeo para cada operación
- **Acoplamiento fuerte** – cambiar la tabla obliga a cambiar el código Java
- **Errores frecuentes** – nombres de columna incorrectos, tipos mal convertidos
- **Difícil mantenimiento** – el código crece y se vuelve ilegible

✓ QUÉ ES ORM B1

ORM (Object-Relational Mapping) automatiza la conversión entre objetos Java y tablas relacionales. El desarrollador define la relación entre clase y tabla, y el sistema genera el SQL automáticamente.

```
// Con ORM, esto:
em.persist(u);
// genera automáticamente:
// INSERT INTO usuarios (nombre, email) VALUES (?, ?)
```

👤 VENTAJAS DEL ORM B1

Abstracción trabajas con objetos, no con SQL.	Menos código elimina mapeo manual repetitivo
Mantenibilidad cambios en modelo más controlados	Productividad desarrollo más rápido

Idea clave: ORM no elimina el SQL ni la BD. Introduce una capa intermedia que traduce automáticamente entre objetos Java y estructuras relacionales. El SQL sigue existiendo internamente.

🔗 ENTIDAD JPA – CLASE MAPEADA B2

```
import jakarta.persistence.*;

@Entity // + obligatorio
@Table(name = "usuarios") // + opcional
public class Usuario {

    @Id // + obligatorio
    @GeneratedValue(strategy =
        GenerationType.IDENTITY) // + AUTO_INCREMENT
    private int id;

    @Column(nullable = false) // + NOT NULL
    private String nombre;

    private String email; // + nombre = columna

    public Usuario() {} // + OBLIGATORIO
    public Usuario(String n, String e){ nombre=n; email=e; }
    // getters y setters...
}
```

⚙️ ENTITYMANAGER – NÚCLEO DEL SISTEMA B2

Es el intermediario entre tu app Java y la BD. Gestiona guardar, recuperar, eliminar y sincronizar objetos. Se crea a partir de `EntityManagerFactory`.

```
EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("miUnidad");
EntityManager em = emf.createEntityManager();
// emf -> objeto pesado, crear UNA sola vez
// em -> objeto ligero, usar para operaciones
```

OPERACIONES BÁSICAS DEL ENTITYMANAGER

Método	SQL generado	Descripción
<code>em.persist(obj)</code>	INSERT	Guarda objeto nuevo en BD
<code>em.find(Clase, id)</code>	SELECT WHERE id	Busca objeto por clave primaria
<code>em.remove(obj)</code>	DELETE	Elimina el objeto de BD
<code>em.merge(obj)</code>	UPDATE	Sincroniza objeto detached
<i>(modificar atributo)</i>	UPDATE	JPA detecta cambios automáticamente
<code>em.close()</code>	–	Cierra el EntityManager

💡 TRANSACCIONES – OBLIGATORIAS AL MODIFICAR

Toda operación que modifique datos (`persist`, `remove`, `update`) debe ir dentro de una transacción. Sin ella los cambios no se guardan.

```
// Estructura obligatoria:
em.getTransaction().begin();
// ... operaciones que modifican BD ...
em.getTransaction().commit(); // confirmar

// En caso de error:
em.getTransaction().rollback(); // deshacer
```

📄 PERSISTENCE.XML B2

```
Archivo de configuración JPA. Ruta exacta: src/main/resources/META-INF/persistence.xml

<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    version="3.0">
    <persistence-unit name="miUnidad">
        <properties>
            <property name="jakarta.persistence.jdbc.url"
                value="jdbc:mysql://localhost:3306/tienda"/>
            <property name="jakarta.persistence.jdbc.user"
                value="root"/>
            <property name="jakarta.persistence.jdbc.password"
                value="1234"/>
            <property name="jakarta.persistence.jdbc.driver"
                value="com.mysql.cj.jdbc.Driver"/>
            <property name="hibernate.hbm2ddl.auto"
                value="update"/>
            <property name="hibernate.show_sql"
                value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

🔄 CICLO DE USO TÍPICO B2

- 1 Crear `EntityManagerFactory` – una vez, al inicio
- 2 Crear `EntityManager` – por operación o sesión
- 3 Iniciar transacción – `em.getTransaction().begin()`
- 4 Realizar operaciones – `persist`, `find`, `remove`, `update`
- 5 Confirmar transacción – `em.getTransaction().commit()`
- 6 Cerrar recursos – `em.close()` · `emf.close()`

🔍 LO QUE OCURRE INTERNAMENTE B2

```
// Escribes esto:
em.persist(usuario);

// Hibernate hace:
// 1. Detecta @Entity y @Column de la clase
// 2. Genera el SQL apropiado
// 3. INSERT INTO usuarios (nombre, email) VALUES (?,?)
// 4. Ejecuta la consulta en MySQL
// 5. Sincroniza el ID generado en el objeto

Regla de oro: define la entidad con @Entity + @Id + constructor vacío · configura persistence.xml · usa em.persist() dentro de transacción · cierra em y emf. El SQL lo genera Hibernate.
```